

Round-Robin Scheduling

Fair preemptive scheduling with a fixed time quantum

Definition

Round-robin scheduling is a preemptive scheduling algorithm in which each runnable process receives the CPU for a fixed time interval called a **quantum**.

1. Main Idea

Fair Rotation

Round robin cycles through the ready processes. Each process gets a turn on the CPU, then the scheduler moves on to the next runnable process.

Quantum

A **quantum** is the maximum amount of time a process may run before the scheduler can preempt it and give the CPU to another process.

Situation	Scheduler Action
Process uses entire quantum	Preempt it and put it at the end of the ready list.
Process blocks before quantum ends	Switch immediately because the process cannot continue.
Process finishes before quantum ends	Switch immediately because the process is done.

2. Ready List Rotation

Implementation

Round robin is easy to implement with a list or queue of runnable processes. The scheduler takes a process from the front, runs it, and if it is still runnable after its quantum, places it at the back.

(a) Before B runs



(b) After B uses its quantum



Key Movement

When B uses its whole quantum, B goes to the end of the list and F becomes the next current process.

3. Round-Robin Scheduler Sketch

```
while ready list is not empty do
  p := remove process from front of ready list

  run p for at most one quantum

  if p finished then
    discard p
  else if p blocked then
    wait for p to become ready again
  else
    add p to the end of the ready list
  end if
end while
```

4. Why Quantum Length Matters

Main Tradeoff

The only really interesting design issue in round robin is choosing the quantum length.

Quantum Choice	Consequence
Too short	Too many context switches, lowering CPU efficiency.
Too long	Poor response time for short requests near the end of the queue.
Reasonable middle	Balances responsiveness with context-switch overhead.

5. Context-Switch Overhead

Context Switch

A **context switch** is the work needed to stop one process and start another, including saving and loading registers, memory maps, tables, and possibly cache state.

Overhead Fraction

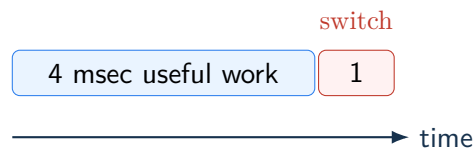
If a process uses its full quantum, the approximate wasted fraction is:

$$\frac{\text{context switch time}}{\text{quantum} + \text{context switch time}}$$

Short Quantum Example

Suppose a context switch takes 1 msec and the quantum is 4 msec. After 4 msec of useful work, the CPU spends 1 msec switching. The overhead is:

$$\frac{1}{4 + 1} = 20\%$$



Too Much Waste

Spending 20% of CPU time on administrative switching overhead is usually too much.

6. Longer Quantum Improves Efficiency

Longer Quantum Example

If the context switch is still 1 msec but the quantum is 100 msec, the overhead is approximately:

$$\frac{1}{100 + 1} \approx 1\%$$

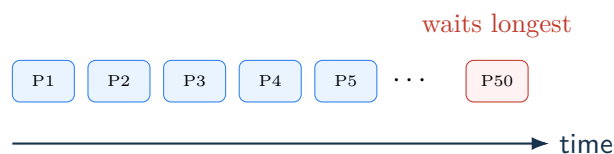
Efficiency Benefit

A longer quantum reduces the percentage of CPU time lost to switching overhead.

7. Long Quantum Hurts Response Time

Server Example

Suppose 50 requests arrive in a short time, and each process may use a full 100 msec quantum. The last process may wait about 5 seconds before it gets its first chance to run.



User Perception

Even if the CPU is efficient, a short interactive request that waits several seconds before starting will feel sluggish.

8. Quantum and CPU Bursts

Mean CPU Burst

If the quantum is longer than the average CPU burst, preemption will not happen often. Most processes will block before their quantum expires.

Why This Helps

When processes block before the quantum ends, context switches happen because they are logically necessary. This avoids unnecessary timer-based preemptions.

Case	Effect
Quantum shorter than many CPU bursts	Frequent preemption and high switching overhead.
Quantum much longer than CPU bursts	Fewer preemptions, but possible poor response under load.
Quantum near a good compromise	Acceptable overhead and acceptable response time.

9. Choosing a Good Quantum

Too Short

A very short quantum gives good responsiveness in theory, but can waste CPU time on constant context switches.

Too Long

A very long quantum behaves more like first-come, first-served scheduling and can make short requests wait behind long ones.

Common Compromise

A quantum around 20–50 msec is often a reasonable compromise between CPU efficiency and response time.

10. Round Robin vs. FCFS

Feature	FCFS	Round Robin
Preemption	Nonpreemptive.	Preemptive after a quantum.
Ready structure	FIFO queue.	Circular ready queue or list.
Long CPU-bound job	Can hold CPU until it blocks.	Loses CPU when its quantum expires.
Short interactive request	May wait behind a long job.	Gets a turn after at most one pass through the queue.
Overhead	Low switching overhead.	Depends strongly on quantum length.

11. Quick Review

Key Points

- Round robin is a preemptive scheduling algorithm.
- Each process gets a fixed time quantum.
- If a process uses its full quantum, it is moved to the end of the ready list.
- If a process blocks or finishes early, the CPU switches immediately.
- Round robin is simple and fair because processes take turns.
- A short quantum improves responsiveness but increases context-switch overhead.
- A long quantum improves CPU efficiency but can hurt response time.
- A 4 msec quantum with a 1 msec switch wastes about 20% of CPU time.
- A 100 msec quantum with a 1 msec switch wastes about 1% of CPU time.
- If the quantum is longer than the average CPU burst, most processes block before preemption.
- A quantum around 20–50 msec is often a reasonable compromise.